

MatrixDSL

Phase 2: Evolving MatrixDSL into a Modern
Tensor / AI Compiler and Accelerator

Project ID	A3
Team Number	13
Review	Review 2 — Architecture, Modernization and Implementation Progress
Review Date	_____
Faculty / Evaluator	_____

TEAM MEMBERS

Name	Register Number	Phase 2 Responsibility
Vinay A	24BCB0131	Lexer, Parser, AST — tensor syntax extensions
Mudpe Parth Tulsidas	24BDS0353	Semantic Analysis — batch shape rules, dtype checking
Kandi Jeevitesh Reddy	24BCE0350	Tensor IR, operator fusion, tiling passes
Shreyas Mandem	24BCE0381	MDT ISA extensions, memory hierarchy, simulator

1. Review of Phase 1

1.1 What is already well designed — KEEP

Decision	Why it holds up
Shape-is-the-type semantic analysis	Turns dimension errors into compile-time type errors; this is the project's core justification and nothing in Phase 2 threatens it
Matrix operations as opaque LLVM intrinsic calls, not loop nests	Survives -O2 intact, lowers to exactly one MDT instruction; this pattern generalises directly to every new fused operation added below
Frozen shared interfaces on <code>main</code> , per-member branches	Let four people work in parallel from week 3; the same discipline is reused for Phase 2
Three disjoint register classes (scalar / vector / matrix)	[FACT] RISC-V International's own Attached Matrix Extension effort and Alibaba's XuanTie MME use the identical philosophy — a matrix register file <i>decoupled</i> from the vector register file, so matrix and vector operations can execute independently. MDT arrived at the same structure independently in Phase 1; Phase 2 keeps it rather than merging the register files.
Differential testing (host vs. MDT, compare element-wise)	Remains the primary correctness oracle in Phase 2, extended to the new operations
C++ / CMake, no scripting-language dependency	Kept as house style
MDT instruction-set simulator as "proof the ISA is executable"	Kept and substantially extended (Section 5)

1.2 What limits MatrixDSL for modern AI — the honest gap list

Limitation	Why it matters
Only 2-D matrices	Batched GEMM, multi-head attention and convolution are all fundamentally ≥ 3 -D. A language that can only express one matrix at a time cannot express a transformer layer.
FP32-only	Every production AI accelerator lives on reduced precision. An "AI compiler" that only understands FP32 is not a credible claim.
No memory hierarchy	The Phase 1 simulator has one flat array. There is no scratchpad/SRAM distinct from DRAM, so nothing about data reuse, blocking or double buffering can be studied — and that is exactly what accelerator-compiler research is about.
No operator fusion	<code>matmul</code> and <code>relu</code> are always two separate calls even when chained back-to-back. This is the single highest-value missing optimization.
No batched execution path	Nothing in the tiling pass understands "do this same 4×4 operation N times for N batch elements."

1.3 What should remain unchanged

- The language stays small: no user-defined functions, no source-level control flow. Internally generated tiling loops are unaffected.
- The six-stage pipeline shape (frontend → shape check → IR → optimizer → backend → assembly) is not replaced, only extended with one new layer (Section 3).
- Differential testing against a reference/host implementation remains the primary correctness oracle.
- Static, compile-time-known shapes only. [OUR DESIGN] Dynamic (runtime-determined) shapes are explicitly rejected for Phase 2 — see §1.4.

1.4 What should NOT be added — and why

This is stated plainly because the temptation in a "modernize toward AI" prompt is to add everything on the list. Four cuts matter most:

Rejected	Reason
Full MLIR adoption as an out-of-tree dialect stack	The engineering cost is the same category of risk Phase 1 already flagged for raw TableGen/SelectionDAG: a real MLIR dialect needs MLIR's build, its ODS/TableGen-adjacent tooling, and its <code>PassManager</code> and dialect-conversion framework, on top of everything already standing. MLIR's payoff — dozens of interoperating dialects reused across many hardware backends — is amortised over large, long-lived projects. For one target, one semester, four people, the payoff does not clear the cost. See §3 for the alternative actually adopted.
Full N-dimensional tensors with general broadcasting	Generalising to arbitrary rank blows up the type system, the shape-rule set, and layout reasoning — exactly the kind of unbounded scope the "do not add everything" instruction is warning against. A single new batch dimension (§2) captures the AI-relevant cases (batched GEMM, attention, per-sample convolution) without this cost.
Dynamic / runtime-determined shapes	Requires symbolic shape reasoning and runtime shape polymorphism throughout the pipeline. Phase 1's entire value proposition rests on shapes being known and checked at compile time; dynamic shapes would quietly undo that.
Real asynchronous DMA, multiple compute units, hardware synchronization	None of this can be verified by a functional simulator without also building a scheduler and a concurrency model — a project on its own. Section 5 models the <i>benefit</i> of overlap analytically instead, which is what the prompt's "lightweight performance model" actually asks for.

2. Modernizing MatrixDSL

2.1 The one language extension: a batch dimension

[OUR DESIGN] MatrixDSL gains exactly one new capability: a batch dimension on top of the existing 2-D matrix.

```
tensor W[4][4];           // unchanged: a plain 2-D matrix (batch = 1)
tensor X[8][4][4];       // NEW: a batch of 8 independent 4x4 matrices
```

`MatrixType` gains one integer field (`batch`, default 1) rather than becoming an arbitrary-rank shape list. Every existing shape rule (§ shape-rules.md from Phase 1) is reused unchanged on the trailing two dimensions; only one new rule is added — batch compatibility — and it is deliberately narrow:

Two tensors are batch-compatible if their batch counts are equal, **or** one of them has batch = 1 (broadcast across the batch).

This is the one piece of broadcasting Phase 2 implements. General NumPy-style broadcasting across arbitrary dimensions is explicitly not implemented (§1.4).

2.2 New builtin: `softmax`

One new builtin function, following the exact shape as `relu` (shape-preserving, unary):

```
tensor scores[8][4][4];
tensor weights[8][4][4];
weights = softmax(scores);    // row-wise softmax, batch-aware
```

No new syntax beyond one keyword. Everything else needed for the two Phase 2 benchmark workloads (§7) — batched matmul, transpose, softmax, relu — is expressible by composing operations MatrixDSL already has, once batching exists. This is a deliberate design win: **attention and a transformer feed-forward block both become expressible with zero additional grammar beyond the batch dimension and `softmax`.**

2.3 Feature-by-feature disposition

Every item the Phase 2 prompt asks us to "consider," classified honestly rather than accepted wholesale:

Feature	Disposition	Reasoning
Batch dimension (3-D: $B \times R \times C$)	ADD	Minimal type-system change, unlocks batched GEMM and attention
Batch broadcasting (batch=1 case only)	ADD	Needed for bias/weight reuse across a batch, narrowly scoped
Full N-D tensors, general broadcasting	DO NOT ADD	Unbounded scope increase, see §1.4
Static shapes	KEEP	Compile-time shape checking is the language's whole premise
Dynamic shapes	DO NOT ADD	See §1.4
Reductions (row-max, row-sum)	ADD	Required primitives for <code>softmax</code> ; scoped to exactly what softmax needs
Layouts (row-major only)	KEEP	Alternate/blocked layouts as a first-class type — [RESEARCH IDEA] , not core
Tiling	IMPROVE	Extended to be batch-aware and memory-hierarchy-aware (§5)
Vectorization	KEEP	Already implicit via <code>VADD</code> / <code>VMUL</code>
Operator fusion	ADD	§6 — the single highest-value addition
Memory spaces (scratchpad)	ADD	§5
Async operations	DO NOT ADD	See §1.4
Batched matmul	ADD	Compiler-level; no new opcode required (§4)
Convolution	OPTIONAL	Only as a compiler-lowered im2col-plus-matmul decomposition, not a new instruction; not a committed Phase 2 deliverable
Attention	ADD	Flagship benchmark, composite lowering only (§7)
Softmax	ADD	Composite lowering: row-max → subtract → exp (runtime call) → row-sum → divide
LayerNorm	OPTIONAL	Decomposes into mean/variance reductions + elementwise; not a committed benchmark
GELU	DO NOT ADD	ReLU remains the one hardware-backed activation; GELU has no clean fixed-point-friendly decomposition worth the effort here

3. IR Design

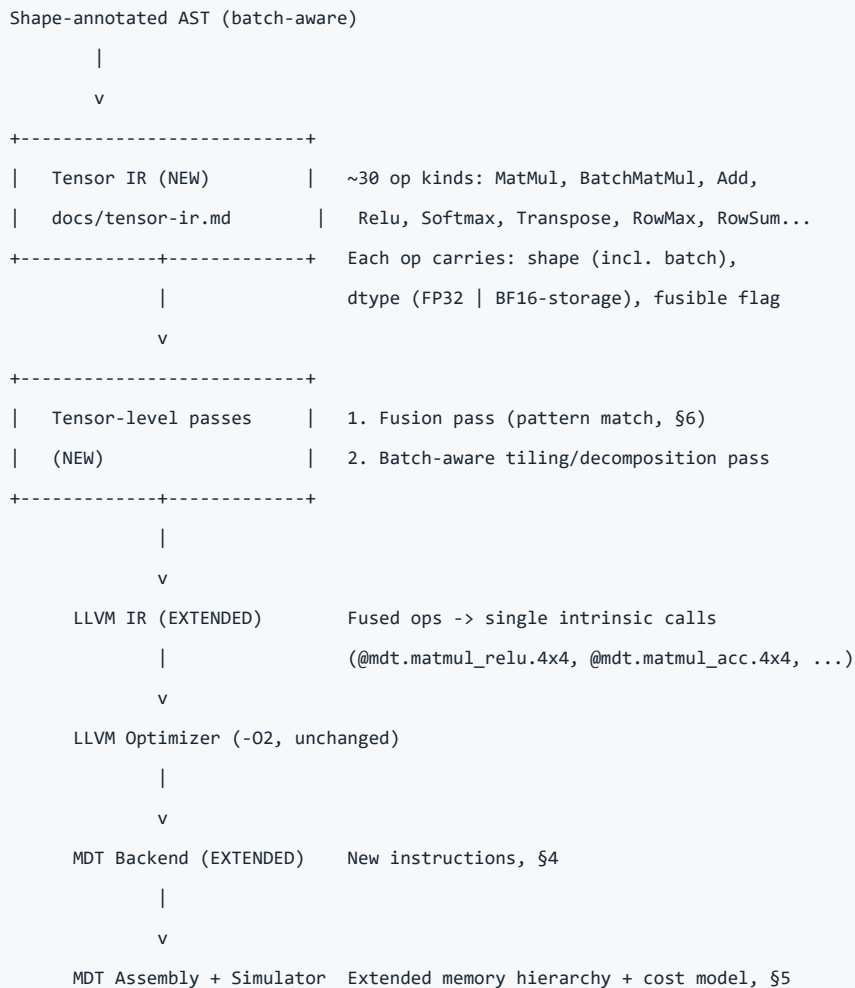
3.1 The two candidate pipelines

Candidate A. MatrixDSL → LLVM IR → MDT (Phase 1, unchanged).

Candidate B. MatrixDSL → high-level Tensor IR → optimization → MDT-specific IR → LLVM IR → MDT backend → MDT assembly.

3.2 Recommendation: Candidate B, implemented as a small hand-rolled IR — not MLIR

[OUR DESIGN] Phase 2 adds exactly one new layer — a **Tensor IR** — between semantic analysis and LLVM IR generation. This is Candidate B's shape, but its *implementation* deliberately does not depend on the MLIR framework.



3.3 Why not MLIR

[EXISTING SYSTEM] The real OpenXLA/IREE stack lowers through the MLIR `linalg` dialect (structured tensor computations) and `vector` dialect (explicit vectorization) before reaching target-specific dialects and finally the `llvm` dialect and LLVM IR — **[FACT]** this progressive-lowering sequence through named dialects is documented IREE compiler behaviour, not a paraphrase. **[EXISTING SYSTEM]** TVM's TensorIR takes a related but distinct approach: it keeps one schedulable IR and applies schedule primitives (`split`, `reorder`, `cache_read`/`cache_write`, `compute_at`) directly to it rather than progressively lowering through several dialects — **[FACT]** these are TVM's actual, named schedule-primitive APIs.

Both are genuine engineering achievements built by teams far larger than four people, sustained over years. **[OUR DESIGN]** MatrixDSL's Tensor IR borrows the *idea* common to both — represent a tensor program at a level where fusion and tiling decisions are legible, before committing to hardware-specific IR — without adopting either framework's machinery. Concretely:

- No dialect-registration system, no `PassManager`, no dialect-conversion framework — Tensor IR is a plain C++ class hierarchy, structurally the same style as the existing `AST.h`.
- Roughly 30 operation kinds, not the dozens-to-hundreds a production dialect accumulates.

- Two passes (fusion, tiling), not an extensible pass pipeline framework.

This is the same judgment call already made once in Phase 1 (direct IR-to-assembly emission was kept as the documented fallback to a full TableGen/SelectionDAG backend). The team is consistently choosing "borrow the idea, skip the framework" when the framework's fixed cost does not fit a one-semester, four-person budget — and consistently keeping the option to point at the real system by name in the report, so the research lineage is honest and traceable.

3.4 What Candidate A alone (unmodified Phase 1 pipeline) would have cost us

Without a Tensor IR, fusion decisions would have to be made either in the AST (too early — shapes aren't fully resolved and batch broadcasting hasn't been applied) or in raw LLVM IR (too late — LLVM's generic optimizer does not know `@mdt.matmul.4x4` followed by `@mdt.relu.4x4` is a fusible pattern specific to MDT; pattern-matching IR-level call sequences post-optimization is fragile in exactly the way Phase 1's `docs/llvm-backend.md` already warned against for loop-idiom recognition). The Tensor IR exists at precisely the point where operations are still named, shaped, and adjacent enough to fuse reliably.

4. MDT Architecture and ISA Extensions

4.1 Register file — unchanged

R0–R15 (scalar), V0–V7 (vector), M0–M7 (matrix, 4×4). **[OUR DESIGN]** Kept exactly as Phase 1 defined it; see §1.1 for why.

4.2 New instructions

[OUR DESIGN] Seven new instructions, chosen to be the minimum set that makes fusion, mixed-precision storage, and softmax possible. 17 (Phase 1) + 7 = **24 total** — still small enough to read end to end.

Instruction	Semantics	Motivation
<code>MATMULACC Md, Ma, Mb</code>	$Md = Md + (Ma \times Mb)$	Accumulate form of MATMUL — the bias/residual-add half of fusion (§6)
<code>MATMULRELU Md, Ma, Mb</code>	$Md = \text{relu}(Ma \times Mb)$	The flagship fused instruction — see §6
<code>CVT.F32.BF16 Rd, Rs</code>	Convert FP32 → BF16 (storage)	Mixed-precision <i>storage</i> , not compute — see §4.3
<code>CVT.BF16.F32 Rd, Rs</code>	Convert BF16 → FP32 (storage)	"
<code>VREDSUM Rd, Vs</code>	$Rd = Vs[0] + Vs[1] + Vs[2] + Vs[3]$	Reduction primitive, feeds softmax's normalisation
<code>MROWMAX Vd, Ms</code>	Row-wise max of a 4×4 matrix into a 4-lane vector	Numerically-stable softmax needs max-subtraction before <code>exp</code>
<code>MROWSUM Vd, Ms</code>	Row-wise sum of a 4×4 matrix into a 4-lane vector	Softmax normalisation denominator

[FACT] MATMULRELU's justification has a direct, citable precedent: XLA's *epilogue fusion* merges an elementwise operation (bias, activation) onto the end of a reduction kernel like matmul, because the output values already sit in the accumulator registers immediately before the store — applying the activation there is essentially free. MATMULACC and MATMULRELU are MDT's hardware-level implementation of exactly that idea.

4.3 Numeric formats — storage-only mixed precision

[OUR DESIGN] Matrix registers (M0 – M7) keep computing in FP32 internally — the MATMUL datapath from Phase 1 is untouched. BF16 is introduced only as a storage/memory format, converted to/from FP32 at load and store time via the two new CVT instructions.

This is a deliberate, narrower claim than "mixed-precision compute." [FACT] Production accelerators generally do reduce memory bandwidth and footprint with lower-precision storage while often accumulating in higher precision internally — the storage/compute precision split is a real and common pattern, not an invented one. Teaching the full compute datapath two number formats (rounding modes, denormals, dual ALU paths) is out of scope; teaching a conversion instruction at the memory boundary is not.

INT8/INT4 are explicitly OPTIONAL / future work, not committed Phase 2 scope — see §1.4's scoping discipline.

4.4 Comparison table

Concept	Real system	MDT Phase 2
Decoupled matrix register file	[FACT] RISC-V Attached Matrix Extension, XuanTie MME	Kept from Phase 1 (M0–M7 separate from V0–V7)
Epilogue/activation fusion	[FACT] XLA epilogue fusion, CUTLASS epilogues	MATMULRELU , MATMULACC
Mixed-precision storage, higher-precision accumulate	[FACT] common accelerator pattern	CVT.F32.BF16 / CVT.BF16.F32 at load/store boundary only
Systolic array scale	[FACT] TPU v6e (Trillium): 256×256; TPU TensorCore: 128×128	MDT: 4×4 (teaching scale, explicitly not claimed equivalent)

MDT is not claimed to be competitive with, or architecturally equivalent to, any of the real systems in this table. The comparison exists to show the *pattern* is shared, not the scale.

5. Memory Hierarchy and Tiling

5.1 The hierarchy

```
DRAM (flat array, Phase 1's only memory – kept)
|
v  SCRATCHLOAD / SCRATCHSTORE (new, compiler-inserted, bulk block copy)
|
Scratchpad / SRAM (NEW – small, fixed-size, separate address space)
|
v  LOAD / STORE (unchanged instructions, now scratchpad-aware)
|
Vector / Matrix registers (unchanged)
```

[OUR DESIGN] A second flat memory array is added to the simulator: a small scratchpad (sized to hold a few dozen 4×4 tiles — proportionate to a *teaching* accelerator, explicitly not claimed to resemble a real accelerator's on-chip memory, which runs to tens or hundreds of megabytes). **[FACT]** For scale contrast: TPU v5e's VMEM scratchpad is 128 MiB. MDT's scratchpad is chosen purely to make blocking and reuse observable in the performance model, not to be realistic in absolute size.

5.2 Addressing — reusing headroom already in the Phase 1 encoding

[OUR DESIGN] Phase 1's instruction encoding (`docs/isa.md` §6) left reserved bits in every instruction format. One reserved bit in the I-type format becomes the address-space selector: 0 = DRAM, 1 = scratchpad. `LOAD / STORE` gain no new opcode — only a new encoding bit — which keeps the instruction count claim in §4.2 honest (no hidden growth).

5.3 DMA, modelled synchronously

[OUR DESIGN] `SCRATCHLOAD / SCRATCHSTORE` are compiler-inserted, not user-visible, bulk block-copy pseudo-instructions that move a tile-block between DRAM and scratchpad. They are **blocking** in the executed simulation — no real concurrency is modelled at execution time. **[FACT]** Real double buffering (a "ping/pong" pair of scratchpad buffers that overlap transfer and compute) is a software-pipelining technique used in real accelerator compilers such as Google's Pallas for TPU. MDT models the *benefit* of double buffering analytically in the performance model (§11) rather than actually overlapping execution — see §1.4 for why real concurrency is out of scope.

5.4 Who decides what

[OUR DESIGN], mirroring the real division of labour in accelerator compiler stacks (**[FACT]** e.g. Pallas' `BlockSpec` + `grid` API, where the compiler describes tiling and the runtime schedules the transfers):

Decision	Owner
Tile size, tile ordering	Compiler (tiling pass)
What data lives in scratchpad, and when	Compiler (tiling pass inserts <code>SCRATCHLOAD</code> / <code>SCRATCHSTORE</code>)
Whether a block is DRAM- or scratchpad-resident at a given instruction	Compiler (encodes the address-space bit)
Actually copying bytes and executing instructions	Hardware / simulator — no hardware-side prefetch heuristics in this design

5.5 Tiling strategy — extended, not replaced

Phase 1's tiling (decompose a large 2-D matrix into a grid of 4×4 tiles) is extended with one nesting level for the batch dimension:

```

for b in 0 .. batch:
  for i in 0 .. M/4:
    for j in 0 .. N/4:
      SCRATCHLOAD the needed A/B tiles for (b,i,j) if not already resident
      accumulate via MATMUL / MATMULACC across k
      SCRATCHSTORE the result tile

```

Register tiling (keeping at most 3 tiles live in M0–M7 per Phase 1's risk register R6) is unchanged. Hierarchical blocking (DRAM → scratchpad → registers) is the one genuinely new tiling concept, and it is the direct enabler of the ablation study in §8.

6. Operator Fusion — one committed approach

[OUR DESIGN] Compiler-driven pattern matching at the Tensor IR level, lowering matched patterns directly to the new fused MDT instructions. Not runtime/library calls (too heavy for an embedded-style target), and not a TVM-style auto-tuned kernel library (an auto-tuning search system is a project on its own).

```
Y = relu(matmul(A, B) + C);
```

Tensor IR before fusion:

```
t1 = MatMul(A, B)
t2 = Add(t1, C)
Y = Relu(t2)
```

Fusion pass recognises: MatMul -> Add(bias) -> Relu

Rewrites to:

```
Y = FusedMatMulBiasRelu(A, B, C)
```

Lowering emits TWO MDT instructions, not three:

```
(load C into Md)
MATMULACC Md, Ma, Mb      ; Md = C + (A x B)
(a separate RELU on Md, since no 3-way fused opcode exists)
```

[OUR DESIGN] Not every fusible pattern gets its own dedicated hardware instruction. The two-operation pattern `matmul → relu` gets `MATMULRELU` (§4.2) because it is the highest-frequency pattern in both benchmark workloads (§7). The three-operation pattern above is fused down to the *smallest available instruction sequence* (2 instructions instead of 3) rather than motivating an eighth new opcode. This is a deliberate ISA-growth discipline: fusion is a compiler technique first, and a hardware instruction only for the single pattern that earns it by frequency.

7. AI Workloads and Benchmarks

[OUR DESIGN] Two workloads, chosen because both are expressible with zero new grammar beyond §2, both exercise every Phase 2 addition, and both are recognisable pieces of a real transformer:

7.1 Benchmark 1 — Transformer feed-forward (MLP) block

```
H = relu(matmul(W1, X));    // fuses to MATMULRELU
Z = matmul(W2, H);
```

Exercises: batched matmul, the fused instruction, and — at the memory-hierarchy level — the classic up-projection/down-projection weight-reuse pattern that motivates scratchpad blocking.

7.2 Benchmark 2 — Scaled dot-product attention (single head, static small sequence)

```
scores = matmul(Q, transpose(K));
weights = softmax(scores);
out     = matmul(weights, V);
```

Exercises: `transpose`, batched matmul, the new composite `softmax` lowering (`MROWMAX` → subtract → `exp` via a runtime call → `MROWSUM` → divide), and is deliberately memory-bandwidth-stressing, which is exactly what the scratchpad/tiling story needs to demonstrate a measurable difference.

Explicitly excluded from Phase 2 benchmarks: KV-cache (an incremental-decoding, multi-call stateful concept — real complexity, no payoff for a static-shape compiler), and quantized INT8/INT4 inference as a hard commitment (kept **OPTIONAL** — see §4.3; a stretch variant compares FP32-stored vs. BF16-stored weights on Benchmark 1 using the memory-traffic column of the performance model, since the **CVT** instructions make that comparison nearly free once built).

8. Research Direction

[OUR DESIGN] The strongest available angle, and the one Phase 2 is scoped around:

A controlled measurement of instruction-count and analytical-cycle reduction from compiler-driven operator fusion and memory-hierarchy-aware tiling, on a fully open, from-scratch DSL/IR/ISA/simulator stack.

This directly answers the prompt's "compiler/architecture co-design" and "memory-aware compilation" pointers with a concrete, executable experiment rather than a survey.

The experiment

Compile both benchmark workloads (§7) under four configurations, changing exactly one variable at a time:

Configuration	Fusion	Scratchpad tiling
(a) Baseline	off	off
(b) Fusion only	on	off
(c) Tiling only	off	on
(d) Full Phase 2 pipeline	on	on

For each configuration, report from the simulator/performance model (§11): MDT instruction count, DRAM↔scratchpad bytes moved, and the analytical cycle-count estimate. Because every number is produced by static analysis of the team's own IR and simulator, no external claim is being borrowed — the result is a small, honest, four-cell ablation that is directly citable in the final report's evaluation section, and is the natural continuation of Phase 1's "compile twice, compare" measurement discipline applied to a new axis (fusion + memory, rather than safety facts).

9. Final Recommended Phase 2 Architecture

One recommended architecture, not several — per the deliverable's own instruction.

9.1 Final compiler pipeline

```
MatrixDSL source (.mtx) [batch-aware tensor declarations, softmax builtin]
|
Lexer / Parser -----> AST                               Vinay
|
Semantic Analysis -----> batch-shape-checked AST         Parth
|
Tensor IR construction -----> Tensor IR (NEW)             Kandi
|
Tensor-level passes:
1. Operator fusion          (matmul->relu, matmul->add)      Kandi
2. Batch-aware hierarchical
   tiling / scratchpad
   insertion
|
LLVM IR Generation -----> LLVM IR (extended intrinsics) Kandi
|
LLVM Optimization -----> optimized LLVM IR              (LLVM, unchanged)
|
MDT LLVM Backend -----> MDT assembly (24 instructions) Shreyas
|
MDT Simulator -----> execution + performance model      Shreyas
```

9.2 MatrixDSL / tensor language changes

Batch dimension (`tensor[B][R][C]`), batch-broadcast, `softmax` builtin. Nothing else in the surface syntax changes. (§2)

9.3 IR design

New Tensor IR layer, hand-rolled (not MLIR), ~30 op kinds, two passes (fusion, tiling). (§3)

9.4 MDT architecture

Register file unchanged (R0–R15, V0–V7, M0–M7). (§4.1)

9.5 MDT ISA extensions

`MATMULACC`, `MATMULRELU`, `CVT.F32.BF16`, `CVT.BF16.F32`, `VREDSUM`, `MROWMAX`, `MROWSUM` — 7 new, 24 total. (§4.2)

9.6 Memory hierarchy

DRAM → scratchpad (new) → registers. `SCRATCHLOAD` / `SCRATCHSTORE` compiler-inserted, synchronous. Address-space bit reuses reserved encoding from Phase 1. (§5)

9.7 Optimization passes

Operator fusion (pattern match → fused instruction or shortest instruction sequence) + hierarchical batch-aware tiling. Both are Tensor-IR-level passes, both team-owned (not LLVM's generic passes). (§6, §5.5)

9.8 Tiling strategy

Three levels: DRAM block → scratchpad-resident tile block → register-resident 4×4 tile, extended with a batch loop at the outermost level. (§5.5)

9.9 Fusion strategy

Pattern-match at Tensor IR level; dedicated hardware instruction only for the highest-frequency pattern (`matmul→relu`); all other fusible patterns lowered to the shortest available instruction sequence rather than motivating new opcodes. (§6)

9.10 AI workloads and benchmarks

Transformer FFN block; single-head scaled dot-product attention. KV-cache and INT8/INT4 explicitly out of committed scope. (§7)

9.11 Lightweight simulator / performance model

[OUR DESIGN] Extend `mdtsim` with: (a) the scratchpad address space and `SCRATCHLOAD` / `SCRATCHSTORE` execution, (b) a static per-instruction latency table (not cycle-accurate — a fixed cost per instruction class, e.g. scalar op = 1, `MATMUL` / `MATMULRELU` / `MATMULACC` = 4, `SCRATCHLOAD` / `SCRATCHSTORE` = bytes-moved / bandwidth-constant), and (c) an analytical overlap estimate: for a tiled loop, report `max(compute_cycles, dma_cycles)` per block instead of their sum when double-buffering would apply, and `compute_cycles + dma_cycles` when it would not — this is the "fake" double buffering promised in §5.3, implemented as arithmetic over the instruction trace rather than as real concurrency. This directly produces the three columns the §8 ablation study needs (instruction count, bytes moved, estimated cycles) from one program.

9.12 Updated repository structure

```
MatrixDSL/
├─ docs/
│   ├── ... (Phase 1 docs, unchanged)
│   ├── tensor-ir.md          NEW – Tensor IR op catalogue and pass design
│   ├── isa-extensions.md     NEW – the 7 new instructions, encoding, rationale
│   ├── memory-hierarchy.md   NEW – scratchpad, DMA model, tiling strategy
│   └─ review2/
│       ├── Review2_Report.md  this document
│       ├── Review2_Report.pdf  PDF submission copy
│       ├── responsibility-matrix.md
│       └─ roadmap.md
├─ compiler/
│   ├── frontend/             extended: batch dimension, softmax token
│   ├── tensor-ir/            NEW – Tensor IR classes + fusion/tiling passes
│   └─ llvm/                  extended: new intrinsic emission
├─ llvm-backend/MDT/          extended: 7 new instructions in TableGen
├─ tools/mdtsim/              extended: scratchpad, performance model
├─ benchmarks/
│   ├── ffn_block.mtx         NEW
│   └─ attention_head.mtx     NEW
└─ tests/
    ├── tensor-ir/            NEW
    └─ fusion/                NEW
```

9.13 12–16 week implementation roadmap

See [docs/review2/roadmap.md](#).

9.14 Four-person team responsibilities

See [docs/review2/responsibility-matrix.md](#).

9.15 Testing and evaluation methodology

Level	Method
Tensor IR construction	Unit tests per op kind, shape propagation including batch
Fusion pass	Pattern-match tests: confirm <code>matmul→relu</code> fuses, confirm non-adjacent or type-mismatched patterns do <i>not</i> fuse (a fusion pass that over-fuses is worse than one that under-fuses)
ISA extensions	Simulator tests per new instruction, same discipline as Phase 1's <code>tests/backend/</code>
Memory hierarchy	Scratchpad round-trip tests; a differential test comparing a tiled run against a non-tiled run of the same workload — results must match exactly, since tiling changes <i>where</i> data lives, not <i>what</i> is computed
End-to-end	Both benchmark workloads compiled and executed in the simulator, verified against the Phase 1-style reference implementation extended for batch/softmax
Research evaluation	The four-configuration ablation study (§8), reported as a table and short analysis

9.16 Potential research-paper direction

Working title: "*Fusion and Hierarchical Tiling for a Co-Designed Tensor Accelerator: A Controlled Ablation on an Open DSL/IR/ISA Stack.*" Positioned as a small, fully reproducible companion to the production systems it draws from (OpenXLA/StableHLO, TVM TensorIR, TPU Pallas) — the contribution is not scale but *legibility*: every number in the ablation study traces to a specific pass, on a stack small enough to read end to end in an afternoon.

10. Master Classification — every proposed feature

Feature	Classification
Shape-is-the-type semantic analysis	KEEP
Opaque intrinsic calls for matrix ops	KEEP
Three disjoint register classes	KEEP
Differential testing methodology	KEEP
Frozen shared interfaces / parallel workstreams	KEEP
4×4 tiling (register level)	KEEP
Tiling strategy	IMPROVE (extended to hierarchical + batch-aware)
Batch dimension (3-D tensors)	ADD
Batch broadcasting (batch=1 only)	ADD
Full N-D tensors, general broadcasting	DO NOT ADD
Dynamic shapes	DO NOT ADD
Reductions (row-max, row-sum)	ADD
Alternate/blocked memory layouts	OPTIONAL (research idea)
Vectorization (VADD/VMUL)	KEEP
Operator fusion (compiler pass)	ADD
Fused instructions (<code>MATMULRELU</code> , <code>MATMULACC</code>)	ADD
Memory spaces (scratchpad/SRAM)	ADD
DMA (synchronous, compiler-inserted)	ADD
Asynchronous DMA / real overlap	DO NOT ADD
Multiple compute units, synchronization	DO NOT ADD
Batched GEMM	ADD (compiler-level, no new opcode)
Convolution	OPTIONAL (im2col decomposition, not core)
Attention (composite lowering)	ADD
Softmax (composite lowering)	ADD
LayerNorm	OPTIONAL
GELU	DO NOT ADD
KV-cache	DO NOT ADD
BF16 storage format + CVT instructions	ADD

Feature	Classification
BF16/mixed-precision <i>compute</i> datapath	OPTIONAL (research idea)
INT8 / INT4	OPTIONAL
Tensor IR (hand-rolled, MLIR-inspired)	ADD
Full MLIR / out-of-tree dialect adoption	DO NOT ADD
Lightweight analytical performance model	ADD
Cycle-accurate simulation	DO NOT ADD
MDT simulator (functional)	KEEP, extended

11. References

1. LLVM Project. *MLIR: Users of MLIR*. <https://mlir.llvm.org/users/>
2. OpenXLA Project. *StableHLO, XLA, Shardy*. (MLIR-based ML compiler ecosystem)
3. IREE Project. *Compilation flow: linalg, vector, target dialects, llvm dialect*. <https://iree.dev/>
4. Apache TVM. *Design and Architecture; TensorIR*. <https://tvm.apache.org/docs/arch/index.html>, https://tvm.apache.org/docs/deep_dive/tensor_ir/index.html
5. Snider, D. and Liang, R. *Operator Fusion in XLA: Analysis and Evaluation*. arXiv:2301.13062, 2023.
6. Google. *How to Think About TPUs — TPU v6e (Trillium) systolic array, VMEM scratchpad*. <https://jax-ml.github.io/scaling-book/tpus/>
7. Google. *Pallas: TPU kernel language — BlockSpec, grid, double-buffered DMA*.
8. RISC-V International. *Enhancing the Future of AI/ML with Attached Matrix Extension*. <https://riscv.org/blog/enhancing-the-future-of-ai-ml-with-attached-matrix-extension/>
9. RISC-V Tech Hub. *Vector-Matrix Extension (VME) Charter*.
10. T-Head / XuanTie. *Matrix Extension (MME) — decoupled matrix register file*.
11. Tillet, P., Kung, H.T. and Cox, D. *Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations*. MAPL, 2019. (carried forward from Review 1)
12. Ragan-Kelley, J. et al. *Halide*. PLDI, 2013. (carried forward from Review 1)
13. Jouppi, N. et al. *In-Datacenter Performance Analysis of a Tensor Processing Unit*. ISCA, 2017. (carried forward from Review 1)
14. Lattner, C. et al. *MLIR: Scaling Compiler Infrastructure for Domain Specific Computation*. CGO, 2021. (carried forward from Review 1)
15. Kung, H.T. and Leiserson, C.E. *Systolic Arrays for VLSI*. 1979. (carried forward from Review 1)

Appendix A — Continuity with Review 1

Every Phase 2 decision in this document is a direct extension of a Phase 1 decision, not a reset:

Phase 1 decision	Phase 2 continuation
Opaque intrinsic calls survive -02	Fused intrinsics (@mdt.matmul_relu.4x4) follow the identical contract
TableGen backend risk → documented direct-emission fallback	Same judgment reused to reject full MLIR adoption in favour of a hand-rolled Tensor IR
Reserved encoding bits in the R/I/M instruction formats	The scratchpad address-space bit uses exactly this headroom
Differential testing (host vs. MDT)	Extended, unchanged in method, to batch/fusion/tiling correctness
4×4 register tile, 3 register classes	Unchanged; validated in the interim by real RISC-V matrix-extension work